# RC4 Algorithm (Symmetric Stream Cipher)

RC4 is a symmetric key cryptographic algorithm. It was created by Ronald Rivest which is why it is named as "Ron's Code", i.e. RC4 algorithm.

The algorithm involves intensive computations and is therefore designed using software tools. It is broken into three steps namely:

1. Key Scheduling Algorithm (KSA)
2. Pseudo-Random Generation Algorithm (PRGA)
3. XOR operation applied to the keystream with plaintext to get ciphertext

Here, the output of KSA is used as the input for PRGA and the output of PRGA is further used as the input for XORing.

RC4 key (say k) length varies from 1 to 256 bytes. It generates a pseudo-random stream of bytes (a key-stream K) using the key k. These keystream bytes are used for encryption by combining it with the plaintext using bit-wise exclusive-or (XOR).

### Key Scheduling Algorithm (KSA)

To generate the keystream, the algorithm makes use of **two arrays S and T** of length 256 each. These arrays are populated as:

i. An array S of 256 elements S[0] to S[255]. Initially, the array is filled with one byte (8 bits) in each element as S[0] = 0, S[1] = 1, S[2] = 2, ... , S[255] = 255. These values 0, 1, 2, 3, 4, …, 255 are called as Initial Vector (IV).

| Array Indices **i** | **S[i]** | Binary Representation of **S[i]** |
|---|---|---|
| 0 | 0 | 00000000 |
| 1 | 1 | 00000001 |
| 2 | 2 | 00000010 |
| 3 | 3 | 00000011 |
| 4 | 4 | 00000100 |
| 5 | 5 | 00000101 |
| 6 | 6 | 00000110 |
| . | . | . |
| . | . | . |
| . | . | . |
| 253 | 253 | 11111101 |
| 254 | 254 | 11111110 |
| 255 | 255 | 11111111 |

This can be coded as:

```
for i = 0 to 255
    S[i] = i
```

ii. Another array T of 256 elements T[0] to T[255]. This array is filled with repeating the **key k (of N elements)**; i.e. the first N elements of T are copied from k and then k is repeated as many times as necessary to fill T.

For Example, if the *key* with 5 elements is "angel" with ASCII value equivalent to 01100001 01101110 01100111 01100101 01101100 in binary. In decimal, it is:
key[0] = 97
key[1] = 110
key[2] = 103
key[3] = 101
key[4] = 108

The array T is filled as:

| Array Indices **i** | **T[i]** | Binary Representation of **T[i]** |
|---|---|---|
| 0 | 97 | 01100001 |
| 1 | 110 | 01101110 |
| 2 | 103 | 01100111 |
| 3 | 101 | 01100101 |
| 4 | 108 | 01101100 |
| 5 | 97 | 01100001 |
| 6 | 110 | 01101110 |
| . . . | . . . | . . . |
| 253 | 101 | 01100101 |
| 254 | 108 | 01101100 |
| 255 | 97 | 01100001 |

This can be coded as:

```
for i = 0 to 255
        T[i] = key[i mod N]
```

T[0] = 97
T[1] = 110
T[2] = 103
T[3] = 101
T[4] = 108
T[5] = 97
T[6] = 110
T[7] = 103
T[8] = 101
T[9] = 108
.
.
.
T[250] = 97
T[251] = 110
T[252] = 103
T[253] = 101
T[254] = 108
T[255] = 97

After the arrays are initialized as given above, the T array is used to produce initial permutation of S. For this purpose, a loop executes, iterating from 0 to 255. In each case, the byte at position S[i] is swapped with another byte in the S array, as per arrangement decided by T[i]. The following logic is used for this:

```
j = 0
for i = 0 to 255
        j = (j + S[i] + T[i]) mod 256
        swap(S[i], S[j])
```

Let us consider first few steps as:

| Iteration No. | i | j | S[i] | T[i] | New j (j + S[i] + T[i]) mod 256 | | Swapped elements S[i] and S[j] | New swapped S elements | |
|---|---|---|---|---|---|---|---|---|---|
| | | | | | Calculation | New j | | Element | New Value |
| 1 | 0 | 0 | 0 | 97 | (0+0+97) mod 256 | 97 | S[0], S[97] | S[0] | 97 |
| | | | | | | | | S[97] | 0 |
| 2 | 1 | 97 | 1 | 110 | (97+1+110) mod 256 | 208 | S[1], S[208] | S[1] | 208 |
| | | | | | | | | S[208] | 1 |
| 3 | 2 | 208 | 2 | 103 | (208+2+103) mod 256 | 57 | S[2], S[57] | S[2] | 57 |
| | | | | | | | | S[57] | 2 |
| 4 | 3 | 57 | 3 | 101 | (57+3+101) mod 256 | 161 | S[3], S[161] | S[3] | 161 |
| | | | | | | | | S[161] | 3 |
| 5 | 4 | 161 | 4 | 108 | (161+4+108) mod 256 | 17 | S[4], S[17] | S[4] | 17 |
| | | | | | | | | S[17] | 4 |
| 6 | 5 | 17 | 5 | 97 | (17+5+97) mod 256 | 119 | S[5], S[119] | S[5] | 119 |
| | | | | | | | | S[119] | 5 |
| 7 | 6 | 119 | 6 | 110 | (119+6+110) mod 256 | 235 | S[6], S[235] | S[6] | 235 |
| | | | | | | | | S[235] | 6 |
| 8 | 7 | 235 | 7 | 103 | (235+7+103) mod 256 | 89 | S[7], S[89] | S[7] | 89 |
| | | | | | | | | S[89] | 7 |
| 9 | 8 | 89 | 8 | 101 | (89+8+101) mod 256 | 198 | S[8], S[198] | S[8] | 198 |
| | | | | | | | | S[198] | 8 |
| 10 | 9 | 198 | 9 | 108 | (198+9+108) mod 256 | 59 | S[9], S[59] | S[9] | 59 |
| | | | | | | | | S[59] | 9 |

After all the swaps, the array S comes out (upper row showing index number i, lower row showing element of array S[i]) to be:

| 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 | 14 | 15 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 101 | 124 | 172 | 10 | 166 | 26 | 46 | 91 | 2 | 137 | 39 | 243 | 253 | 25 | 3 | 30 |
| 16 | 17 | 18 | 19 | 20 | 21 | 22 | 23 | 24 | 25 | 26 | 27 | 28 | 29 | 30 | 31 |
| 47 | 238 | 196 | 38 | 94 | 149 | 15 | 32 | 248 | 51 | 158 | 150 | 106 | 183 | 67 | 219 |
| 32 | 33 | 34 | 35 | 36 | 37 | 38 | 39 | 40 | 41 | 42 | 43 | 44 | 45 | 46 | 47 |
| 95 | 177 | 138 | 152 | 13 | 188 | 118 | 108 | 207 | 151 | 41 | 142 | 236 | 103 | 55 | 72 |
| 48 | 49 | 50 | 51 | 52 | 53 | 54 | 55 | 56 | 57 | 58 | 59 | 60 | 61 | 62 | 63 |
| 20 | 244 | 216 | 14 | 168 | 90 | 4 | 42 | 153 | 64 | 250 | 129 | 97 | 225 | 87 | 199 |
| 64 | 65 | 66 | 67 | 68 | 69 | 70 | 71 | 72 | 73 | 74 | 75 | 76 | 77 | 78 | 79 |
| 204 | 100 | 16 | 249 | 191 | 82 | 43 | 131 | 24 | 169 | 69 | 54 | 96 | 0 | 255 | 84 |
| 80 | 81 | 82 | 83 | 84 | 85 | 86 | 87 | 88 | 89 | 90 | 91 | 92 | 93 | 94 | 95 |
| 1 | 143 | 242 | 123 | 21 | 93 | 61 | 102 | 224 | 107 | 109 | 79 | 80 | 23 | 229 | 6 |
| 96 | 97 | 98 | 99 | 100 | 101 | 102 | 103 | 104 | 105 | 106 | 107 | 108 | 109 | 110 | 111 |
| 156 | 181 | 105 | 159 | 33 | 141 | 0 | 104 | 9 | 56 | 233 | 178 | 127 | 111 | 135 | 206 |
| 112 | 113 | 114 | 115 | 116 | 117 | 118 | 119 | 120 | 121 | 122 | 123 | 124 | 125 | 126 | 127 |
| 202 | 128 | 31 | 71 | 211 | 222 | 45 | 66 | 163 | 189 | 167 | 201 | 232 | 17 | 251 | 198 |
| 128 | 129 | 130 | 131 | 132 | 133 | 134 | 135 | 136 | 137 | 138 | 139 | 140 | 141 | 142 | 143 |
| 170 | 155 | 115 | 57 | 228 | 98 | 190 | 76 | 59 | 239 | 37 | 147 | 180 | 240 | 197 | 200 |
| 144 | 145 | 146 | 147 | 148 | 149 | 150 | 151 | 152 | 153 | 154 | 155 | 156 | 157 | 158 | 159 |
| 19 | 0 | 213 | 99 | 125 | 44 | 195 | 164 | 176 | 121 | 220 | 212 | 86 | 186 | 34 | 214 |
| 160 | 161 | 162 | 163 | 164 | 165 | 166 | 167 | 168 | 169 | 170 | 171 | 172 | 173 | 174 | 175 |
| 230 | 254 | 40 | 203 | 194 | 231 | 162 | 226 | 187 | 116 | 208 | 22 | 68 | 88 | 192 | 140 |
| 176 | 177 | 178 | 179 | 180 | 181 | 182 | 183 | 184 | 185 | 186 | 187 | 188 | 189 | 190 | 191 |
| 205 | 234 | 119 | 83 | 136 | 63 | 12 | 112 | 217 | 154 | 184 | 81 | 70 | 35 | 174 | 78 |
| 192 | 193 | 194 | 195 | 196 | 197 | 198 | 199 | 200 | 201 | 202 | 203 | 204 | 205 | 206 | 207 |
| 241 | 179 | 210 | 215 | 49 | 144 | 130 | 48 | 133 | 7 | 209 | 92 | 73 | 193 | 28 | 75 |
| 208 | 209 | 210 | 211 | 212 | 213 | 214 | 215 | 216 | 217 | 218 | 219 | 220 | 221 | 222 | 223 |
| 117 | 223 | 50 | 113 | 114 | 148 | 173 | 29 | 53 | 160 | 8 | 139 | 246 | 65 | 252 | 161 |
| 224 | 225 | 226 | 227 | 228 | 229 | 230 | 231 | 232 | 233 | 234 | 235 | 236 | 237 | 238 | 239 |
| 221 | 185 | 27 | 36 | 11 | 110 | 237 | 165 | 5 | 182 | 145 | 171 | 120 | 157 | 134 | 175 |
| 240 | 241 | 242 | 243 | 244 | 245 | 246 | 247 | 248 | 249 | 250 | 251 | 252 | 253 | 254 | 255 |
| 122 | 58 | 235 | 52 | 62 | 126 | 85 | 60 | 132 | 74 | 245 | 227 | 218 | 89 | 247 | 146 |

Permuted S Array

All this makes it up to Key Scheduling Algorithm (KSA) as:

```
for i from 0 to 255
    S[i] := i
    T[i] := key[i mod keylength]
endfor
j := 0
for i from 0 to 255
    j := (j + S[i] +T[i]) mod 256
    swap values of S[i] and S[j]
endfor
```

Once this has been completed, the stream of bytes is generated using the pseudo-random generation algorithm (PRGA).

### Pseudo-Random Generation Algorithm

In this step, the output of *Key Scheduling Algorithm* is used further. S[i] with another byte in S array are swapped as per mechanism decided by the implementation of S. The loop goes from 0 to 255 and then restarts from 0 until the required length of keystream is generated. The logic for *Pseudo Random Generation Algorithm* (PRGA) is as follows:

```
i = 0
j = 0
```

```
Until the required keystream bytes have been generated:
    i = (i + 1) mod 256
    j = (j + S[i]) mod 256
    swap ( S[i], S[j] )
    k = S[(S[i] + S[j]) mod 256]
   output the keystream byte k
end
```

Let us consider first few steps as:

| Iteration No. | Previous i | Previous j | New i (i+1)mod256 | New j (j+S[i])mod256 | Swapped Values of S[i], S[j] | | k = S[(S[i] + S[j]) mod 256] | |
|---|---|---|---|---|---|---|---|---|
| | | | | | | | S[i] + S[j] | k |
| 1 | 0 | 0 | 1 | 124 | S[1] | 232 | 356 | 33 |
| | | | | | S[124] | 124 | | |
| 2 | 1 | 124 | 2 | 40 | S[2] | 172 | 379 | 201 |
| | | | | | S[225] | 207 | | |
| 3 | 2 | 225 | 3 | 50 | S[3] | 10 | 226 | 27 |
| | | | | | S[50] | 216 | | |
| 4 | 3 | 50 | 4 | 216 | S[4] | 166 | 219 | 139 |
| | | | | | S[216] | 53 | | |
| 5 | 4 | 216 | 5 | 242 | S[5] | 26 | 261 | 235 |
| | | | | | S[242] | 235 | | |
| 6 | 5 | 242 | 6 | 32 | S[6] | 46 | 141 | 240 |
| | | | | | S[32] | 95 | | |
| 7 | 6 | 32 | 7 | 123 | S[7] | 91 | 292 | 13 |
| | | | | | S[123] | 201 | | |
| And so on until the required length of keystream is fetched. | | | | | | | | |

Pseudo-Random Generation Algorithm

Thus, the keystream K generated is:

| Index | K[index] | Binary Representation |
|---|---|---|
| 0 | 33 | 00100001 |
| 1 | 201 | 11001001 |
| 2 | 27 | 00011011 |
| 3 | 139 | 10001011 |
| 4 | 235 | 11101011 |
| 5 | 240 | 11110000 |
| 6 | 13 | 00001011 |
| and so on | ..... | ..... |

Generated Keystream

PRGA can be coded as:

```
i := 0
j := 0
while GeneratingOutput:
    i := (i + 1) mod 256
    j := (j + S[i]) mod 256
    swap values of S[i] and S[j]
    k := S[(S[i] + S[j]) mod 256]
```

```
    output k
endwhile
```

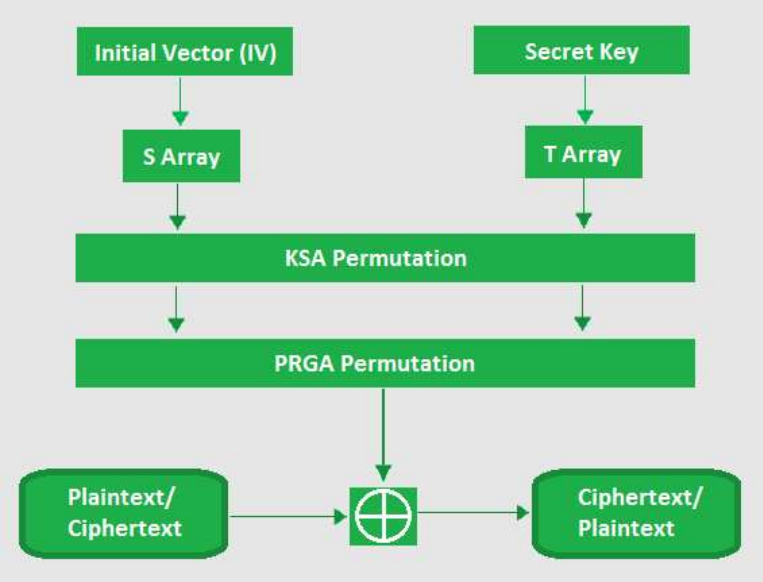
**XORing the keystream**

The keystream K (sequence of bytes ‘k’ given as output by the above PRGA algorithm) generated through this is then XORed with plaintext for encryption. For decryption, the same keystream generated at receiver's end is XORed with ciphertext to get plaintext.

Eg. Let first byte of plaintext is 11110101 and the first byte of keystream is 00100001. Then the XORing takes place at:

11110101 ⊕ 00100001 = 11010100
where the output 11010100 is the ciphertext.

Diagrammatically, RC4 can be shown as:




RC4 Algorithm